C++ Interview Questions

Basic to Advanced, with Answers

- How to use this
- Section 1 — Fundamentals
- Section 2 — OOP and Classes
- Section 3 — Memory and Resource Management
- Section 4 — STL
- Section 5 — Concurrency
- Section 6 — Templates
- Section 7 — Performance and Systems
- Section 8 — Floating Point
- Section 9 — Short Coding Problems
- Section 10 — Questions to Ask Them

How to use this

Questions are graded **[B]** basic, **[I]** intermediate, **[A]** advanced. Answers are written as you would *say* them, not as a textbook paragraph — say the short version first, then expand only if the interviewer wants more.

Where a question relates to something in your assessment, that is flagged, because a worked example from your own code is worth far more than a definition.

Section 1 — Fundamentals

[B] What is the difference between a pointer and a reference?

A reference must be bound at initialisation and cannot be rebound; a pointer can be null and reassigned. A reference is essentially an alias, and the compiler usually implements it as a pointer under the hood, but the language guarantees it always refers to something.

Use a reference when the thing must exist; a pointer when absence is meaningful.

[B] What does `const` mean in these positions?

```
const int* p;        // pointer to const int   -- can't change *p
int* const p;        // const pointer to int   -- can't change p
const int* const p;  // both
int f() const;       // member function doesn't modify the object
```

Read right to left. `const` binds to what's on its left, unless it's leftmost.

Follow-up you'll get: *"What is `mutable` for?"* — it lets a member change inside a `const` member function. The legitimate use is things that aren't part of the logical state: a mutex, a cache, a lazily-computed value. That's why the assessment's cache has `mutable std::mutex` — locking isn't a logical mutation.

[B] What is the difference between `struct` and `class`?

Default access only — `struct` is public, `class` is private. That's it. Convention is `struct` for plain data aggregates, `class` when there's an invariant to maintain.

[B] Stack vs heap?

Stack: automatic lifetime, LIFO, allocation is a pointer bump so it's ~free, size limited (~1-8 MB), and it's almost always cache-warm. Heap: manual or smart-pointer lifetime, large, but allocation may take a lock and may page-fault.

The low-latency version of this answer: heap allocation is banned on a hot path — not for the average cost but for the tail. `malloc` can take a lock and a first touch can page-fault, which is a microsecond-scale outlier in a nanosecond-scale function.

[B] What is `static` in its three meanings?

1. At file scope — internal linkage, not visible to other translation units.
2. Inside a function — one instance, lifetime of the program, initialised on first use (thread-safe since C++11).
3. In a class — belongs to the class, not to any instance.

[B] What happens if you `delete` a pointer twice?

Undefined behaviour, typically heap corruption. This is one of the reasons `unique_ptr` exists — ownership is explicit, so double-delete becomes hard to write.

[I] What is undefined behaviour and why does it matter more than you'd think?

UB means the standard imposes no requirements at all — not "an unspecified value," but *no meaning*.

The practical consequence: **the compiler optimises assuming UB doesn't occur.** So a null check placed after a dereference gets deleted, because the dereference proved the pointer non-null. The symptom often appears far from the cause, and only at high optimisation levels.

The list worth being able to recite: signed overflow, out-of-bounds access, dereferencing null or dangling, reading uninitialised memory, data races, strict aliasing violations, violating strict weak ordering in `std::sort`.

From the assessment: NaN in a `std::sort` comparator is UB, not a bad order. Measured effect — 72 NaNs among 500 elements left 25 perfectly valid entries out of order, because the sort's internal invariants broke.

[I] Signed vs unsigned overflow?

Unsigned wraps — defined behaviour, modulo 2^n. Signed overflow is **undefined**.

The practical trap is unsigned wraparound in index arithmetic:

```
size_t i = 0;
i - 1;                  // 18446744073709551615, not -1
```

From the assessment: the ring buffer index needs `+ size` before the modulo, precisely because `head - count` can go "negative" on an unsigned type and produce a valid-looking but wrong index — silent corruption, not a crash.

```
buf[(head + size - count + i) % size];
```

[I] What is the strict aliasing rule?

You may not access an object through a pointer of an unrelated type. The compiler assumes two pointers of different types don't point at the same memory, which lets it keep values in registers.

The legal way to reinterpret bits is `memcpy` or, since C++20, `std::bit_cast`. A `char*` may alias anything.

Section 2 — OOP and Classes

[B] What is the rule of three / five / zero?

Rule of 0: if your class owns no raw resources, declare none of the special members. Best default.

Rule of 3: if you need a destructor, you almost certainly need a copy constructor and copy assignment too.

Rule of 5: with move semantics, add move constructor and move assignment.

The trap: declaring a destructor *suppresses* the implicit move operations, so your class silently falls back to copying. That's a real performance bug that compiles fine.

[B] When do you need a virtual destructor?

Whenever an object may be deleted through a base-class pointer. Without it the derived destructor never runs — undefined behaviour, and in practice a leak.

The other half of the answer: don't add one if the class isn't meant to be a base. It forces a vtable pointer into every instance, which for a small type can double its size.

[I] How do virtual functions work, and what do they cost?

Each polymorphic class has a vtable — an array of function pointers. Each object holds a pointer to its class's vtable. A virtual call loads the vtable pointer, loads the function pointer, then makes an indirect call.

Cost: two dependent loads plus a branch the predictor can miss. But the bigger cost is that it **blocks inlining**, so all downstream optimisation across that call is lost.

Alternatives: CRTP for compile-time dispatch, `std::variant` + `std::visit` for a closed set of types with no heap, function pointers.

[I] Explain the Liskov Substitution Principle with a concrete violation.

A derived class must be usable anywhere the base is, without the caller knowing the difference.

The classic violation is `Square : Rectangle`. `Rectangle::setWidth` shouldn't change the height — but for a `Square` it must, so code written against `Rectangle` breaks. The types are related mathematically but not behaviourally.

[I] Composition or inheritance — how do you choose?

Inheritance couples you to the base's *implementation*, so a base change can silently break derived classes. That's the fragile base class problem, and it gets worse with depth.

My rule: inherit for interfaces (pure abstract bases), compose for behaviour. And never inherit from STL containers — they have no virtual destructor.

[A] What is the diamond problem and how does virtual inheritance solve it?

If B and C both derive from A, and D derives from both, D has two A subobjects. Ambiguity, and duplicated state.

`virtual` inheritance makes B and C share one A subobject. The cost is an extra indirection to reach the virtual base, and construction order becomes non-obvious — the most-derived class constructs the virtual base.

In practice: prefer a single implementation base plus multiple pure-interface bases, which avoids the problem entirely.

[A] What is PIMPL and when is it worth it?

Put the private members behind an opaque pointer, so the header exposes nothing:

```
class Widget {
public:
    Widget();
    ~Widget();                    // must be defined in the .cpp
    void doThing();
private:
    struct Impl;
    std::unique_ptr<Impl> pimpl_;
};
```

Buys: compilation firewall, stable ABI, hidden dependencies. **Costs:** a heap allocation and a pointer indirection per call — so not for hot-path types.

The destructor must be defined in the `.cpp` where `Impl` is complete, otherwise `unique_ptr` can't destroy an incomplete type.

Section 3 — Memory and Resource Management

[B] What is RAII?

Acquire the resource in the constructor, release it in the destructor. The destructor runs on *every* exit path — normal return, early return, exception — so you cannot leak by forgetting.

`std::lock_guard` is the canonical example: you can't forget to unlock, even if the code throws.

[B] `unique_ptr` vs `shared_ptr` — which and why?

`unique_ptr` for sole ownership. It's **zero overhead** — same size as a raw pointer, no runtime cost. Default choice.

`shared_ptr` only when ownership genuinely is shared. The reference count is **atomic**, so copying one costs a `lock`-prefixed instruction. There are also two allocations unless you use `make_shared`, which combines the control block with the object.

The practical point: passing `shared_ptr` by value in a hot loop is a real cost. Pass by `const&`, or pass a raw pointer/reference when you're not transferring ownership.

[I] What is `weak_ptr` for?

Breaking `shared_ptr` cycles. Two objects holding `shared_ptr` to each other never reach zero and never get destroyed. A `weak_ptr` doesn't contribute to the count; you call `lock()` to get a `shared_ptr` if the object is still alive.

Also useful for caches — observe without keeping alive.

[I] Explain move semantics.

An lvalue has a name and an address; an rvalue is a temporary. `std::move` doesn't move anything — it's a **cast to rvalue reference**, telling overload resolution "you may steal from this."

```
std::vector<int> b = std::move(a);   // b takes a's buffer; a is valid but unspecified
```

The moved-from object must be left in a valid state — you may destroy it or assign to it, but shouldn't rely on its value.

[I] What is RVO, and why is `return std::move(v)` wrong?

Return Value Optimisation constructs the return value directly in the caller's storage, so there's no copy *and* no move. Since C++17 it's guaranteed for pure prvalues.

```cpp
return v;              // NRVO: constructed in place, zero cost
return std::move(v);   // WRONG: turns a named local into an rvalue expression,
                       // which DEFEATS the elision. You get a move where you'd
                       // have had nothing.
```

[A] What is a forwarding reference and how does `std::forward` work?

```cpp
template <typename T>
void wrapper(T&& arg) {              // forwarding reference, NOT an rvalue ref
    inner(std::forward<T>(arg));
}
```

`T&&` in a **deduced** context is a forwarding reference. Reference collapsing means if you pass an lvalue, `T` deduces to `T&` and `T&&` collapses to `T&`; pass an rvalue and `T` deduces to `T`.

`std::forward<T>` casts back to the original value category, so the lvalue-ness or rvalue-ness of the caller's argument is preserved. `std::move` would unconditionally make it an rvalue, which would be wrong for lvalue arguments.

[A] What is the small string optimisation?

`std::string` stores short strings (typically ≤15 chars on libstdc++) inline in the object itself, avoiding a heap allocation entirely. Longer strings allocate.

Why it matters here: an NSE option symbol like `NIFTY25SEP24000CE` is 17 characters, so it's *past* the SSO threshold — every construction of that string is a heap allocation. That's exactly why the assessment used transparent hashing to look up by `string_view` without constructing a `std::string`.

Section 4 — STL

[B] `vector` vs `list` — when would you use `list`?

Almost never. `vector` wins on nearly everything because of cache locality — a linear scan of a vector is one cache line at a time with perfect prefetching, while a list is a pointer chase with a likely cache miss per element.

`list` is justified only when you need O(1) splice, or stable references across insertion, and you have measured that it matters.

Big-O hides constants: linear search of a 100-element vector typically beats a `map` lookup.

[B] When are `vector` iterators invalidated?

On any reallocation — `push_back` past capacity, `resize`, `insert`. `erase` invalidates from the erase point onward. `reserve()` up front prevents the reallocation case.

[I] How does `unordered_map` work, and when does it rehash?

An array of buckets; each bucket is a chain of separately allocated nodes. When `size / bucket_count` would exceed `max_load_factor` (default 1.0), it allocates a bigger bucket array and **relinks every node**.

The invalidation rule to know precisely:

	Survives a rehash?
References to mapped values	**Yes**
Pointers to elements	**Yes**
Iterators	**No**
A traversal in progress	**No**

References survive because it's **node-based** — each element has its own allocation that never moves. Only the bucket pointer array is rebuilt.

From the assessment: that distinction is exactly what made the race subtle. `auto& buf = buffers_[sym]` stays valid, so the code *looks* safe. The race is in the bucket-chain walk needed to *reach* it — a reader following `->next` pointers while they're being rewritten.

[I] `operator[]` vs `find` on a map — why does it matter?

`operator[]` **default-constructs and inserts** if the key is absent. So it mutates the container on what looks like a read, and it can't be `const`.

From the assessment, this was the primary bug in two problems: a calculation thread asking for a price it had never seen silently added a blank entry, then called `.back()` on an empty vector. The map grew without bound from typos and retired symbols, and `average()` returned `0.0/0` = NaN, which propagates silently because every comparison against NaN is false.

Use `find` on read paths. Always.

[I] What algorithm does `std::sort` use, and is it stable?

Introsort: quicksort, switching to heapsort at depth 2·log n to guarantee O(n log n), plus insertion sort for small ranges.

It is not stable. Equal elements come out in an order determined by pivoting.

From the assessment: that's a real bug in a ranking engine. On a quiet day dozens of stocks sit at exactly 0.00% return, so their relative order differs between the batch run and the daily run — and then you can never reconcile live output against a backtest. Fix: make the comparator a total order by breaking ties on a stable id.

[I] What is strict weak ordering and what happens if you violate it?

The comparator must be irreflexive, antisymmetric, transitive, and have transitive equivalence. Violating it is **undefined behaviour** in `std::sort` — not merely a wrong order. In some implementations it reads past the end of the array.

NaN violates it: `a < NaN` is false and `NaN < a` is false, so they look "equivalent" — but two different values both equivalent to NaN are not equivalent to each other. Transitivity of equivalence fails.

[I] When would you use `partial_sort` or `nth_element`?

When you need the extremes, not a full order.

- `partial_sort` — top k in O(n log k)
- `nth_element` — the kth element in O(n), rest partitioned but unordered

From the assessment: the spec asked for "top gainers and losers," not a total order over 500 stocks. Using `partial_sort` + `nth_element` was a **2.4× win** — more than parallelising the arithmetic could ever have delivered, with no threads and no races.

[A] What are transparent comparators?

Marking your hash and equality with `is_transparent` lets you look up with a type that's merely *comparable* to the key, rather than converting to the key type.

```
struct SvHash {
    using is_transparent = void;
    size_t operator()(std::string_view s) const { return std::hash<std::string_view>{}(s); }
};
```

Without it, `map.find(some_string_view)` constructs a temporary `std::string` — a heap allocation per lookup for a 17-character symbol.

The gotcha: you need `is_transparent` on **both** the hash and the equality functor, or it silently falls back to constructing the key.

[A] `emplace_back` vs `push_back`?

`emplace_back` forwards its arguments to construct the element in place, avoiding a temporary. `push_back` takes an already-constructed object.

If you already have the object, they're equivalent (both move it). The win is only when you'd otherwise construct a temporary just to pass it.

A trap: `emplace_back` uses direct initialisation, so it can call `explicit` constructors that `push_back` would reject. Slightly less type-safe.

Section 5 — Concurrency

[B] What is a data race?

Two threads access the same memory, at least one writes, with no synchronisation between them.

The part to emphasise: it's **undefined behaviour**, not "you might see a stale value." So "what value will I read?" is already the wrong question — the program has no defined meaning. The compiler may hoist your loop condition into a register, split a write in two, or delete code it proves unreachable.

[B] Why isn't `x++` thread-safe on an `int`?

It's three operations — load, add, store. Two threads can both load the same value, both add one, and both store the same result. One increment is lost.

[B] Is `volatile` enough for thread safety?

No. `volatile` stops the compiler eliding the access, which is what you want for memory-mapped hardware. It gives you **neither atomicity nor ordering**, so a shared `volatile int` is still a data race.

The confusion comes from Java and C#, where `volatile` *does* imply ordering. In C++ it doesn't.

[I] Explain acquire-release ordering.

They work as a **pair on the same variable**.

A **release store** means: everything I wrote before this, in program order, becomes visible to anyone who acquire-loads this variable.

An **acquire load** means: I can now see everything the releasing thread did before its store.

```
// Thread A                                    // Thread B
data = 42;                                     if (ready.load(acquire)) {
ready.store(true, memory_order_release);           use(data);   // guaranteed 42
                                               }
```

Without it, `ready = true` could become visible before `data = 42`.

[I] What do the memory orderings cost?

On x86-64: relaxed, acquire and release are all plain `mov` instructions — **free**, because x86 already won't reorder store-store or load-load. Only `seq_cst` stores cost anything (`mfence` or `xchg`, ~20–30 cycles), and RMW operations cost a `lock` prefix.

On ARM it's different — weakly ordered, so acquire/release emit real `ldar` / `stlr` instructions. Code that looks free on x86 costs on Graviton or Apple Silicon.

My default: `seq_cst` unless I've identified a specific pairing, then acquire/release. Relaxed only when I genuinely need nothing but atomicity, like a statistics counter.

[I] `compare_exchange_weak` vs `strong`?

Both mean "if the value equals `expected`, set it to `desired`." `weak` may fail **spuriously** — return false even when the values matched — because on LL/SC architectures the reservation can be lost for unrelated reasons.

`weak` is cheaper, so use it inside a retry loop where a spurious failure just costs one more iteration. Use `strong` when you're not looping.

[I] Is `shared_mutex` the right choice for one writer and many readers?

Usually not, and this catches people out. Acquiring a *shared* lock still **writes** to a shared counter — so N readers on N cores ping-pong one cache line and serialise on coherence traffic. For short critical sections it's often slower than a plain mutex, and it has implementation-defined writer starvation.

It only pays when the read critical section is long enough to amortise that atomic RMW.

[A] Is a mutex enough to make a class thread-safe?

Not necessarily, and this is the point I'd make. **A mutex gives mutual exclusion but not order.** If the computation is order-dependent, you prevent corruption without making the result deterministic.

Measured on the assessment: the same fills across four threads under one mutex produced **30 distinct results from 30 identical runs**, on a single-core machine. Share counts agreed every time (addition commutes) while average prices didn't — so a smoke test checking positions passes cleanly while the money numbers are wrong.

The fix is structural: shard by key so each key is owned by exactly one thread. Order is preserved by construction, the result matches a serial replay, and there's no lock on the hot path at all. And the mutex bought no throughput to compensate — 34, 38, 36 ns/fill on 1, 2 and 4 threads, because the critical section was ~20 ns and contention cost more than the work.

[A] What is false sharing and how do you detect it?

Two threads writing different variables that happen to share a 64-byte cache line. No logical sharing, but the line ping-pongs between cores on every write.

Fix with `alignas(64)` or padding. Detect with `perf c2c`.

One subtlety: don't use `std::hardware_destructive_interference_size` for the padding. Its value varies with compiler version and `-mtune`, so it changes your struct layout — an ABI break from a compiler upgrade. GCC warns about exactly this. Pin the constant.

[A] Write a lock-free SPSC queue.

```cpp
template <typename T, size_t N>           // N must be a power of two
class SpscQueue {
    static_assert((N & (N - 1)) == 0);
public:
    bool push(const T& item) {
        size_t w = write_.load(std::memory_order_relaxed);   // only we write it
        size_t next = (w + 1) & (N - 1);
        if (next == read_.load(std::memory_order_acquire)) return false;  // full
        buffer_[w] = item;
        write_.store(next, std::memory_order_release);       // publish
        return true;
    }
    bool pop(T& out) {
        size_t r = read_.load(std::memory_order_relaxed);
        if (r == write_.load(std::memory_order_acquire)) return false;    // empty
        out = buffer_[r];
        read_.store((r + 1) & (N - 1), std::memory_order_release);
        return true;
    }
private:
    alignas(64) std::atomic<size_t> write_{0};
    alignas(64) std::atomic<size_t> read_{0};
    T buffer_[N];
};
```

Points to make as you write it: each index has exactly one writer so its own load can be relaxed; release publishes the data written before it; acquire makes the other side's data visible; `alignas(64)` on both indices or producer and consumer false-share; power-of-two turns modulo into a mask.

[A] How do you confirm a race is actually fixed?

"It stopped reproducing" is not evidence — a race made 100× rarer looks identical to one that's fixed, right up until it isn't.

In order of strength:

1. **Compile-time annotations.** Clang `-Wthread-safety` with `GUARDED_BY` turns "field accessed without its lock" into a compile error. That's a proof over all paths, not a sample.
2. **Model checking** for lock-free code — explores all interleavings the memory model permits.
3. **ThreadSanitizer** — finds races on paths that execute, whether or not they manifest. Caveat: it **cannot model standalone** `atomic_thread_fence`, so a fence-based seqlock can't be validated this way.
4. **Stress testing** as corroboration only — threads oversubscribed 2–4×, recorded seeds so failures replay, and run in both optimised and instrumented builds because they find different things.

And run every check against the **old** code first. If TSan doesn't flag the original, your harness isn't reaching the racy path, and its silence on the new code means nothing.

Section 6 — Templates

[I] When would you use a template instead of a virtual function?

When the type is known at compile time and the call is hot enough that inlining matters. A virtual call blocks inlining, so you lose all downstream optimisation across it; a template resolves at compile time so the optimiser sees straight through.

Always give the cost too: code bloat (one instantiation per type, so I-cache pressure), slower compiles, everything in headers, worse error messages, and — the practical one — **you can't read the parameter from a config file.**

From the assessment: the ranking engine's window size is a runtime `Config` field rather than a template parameter, precisely so it can be configured per deployment.

[I] What is CRTP and what does it solve?

Static polymorphism — an interface-like structure with dispatch resolved at compile time.

```
template <typename Derived>
class HandlerBase {
public:
    void handle(const Tick& t) {
        static_cast<Derived*>(this)->handleImpl(t);   // compile-time
    }
};
class MyHandler : public HandlerBase<MyHandler> { /* handleImpl */ };
```

The base knows its derived type as a parameter, so the call inlines.

Cost: there's no common base type, so no `vector<HandlerBase*>` — you can't hold heterogeneous handlers in a container.

[I] What is `if constexpr` and what did it replace?

Compile-time branching. The untaken branch isn't compiled at all, so it may contain code that would be invalid for that type.

It replaced most uses of SFINAE, which existed only to remove overloads from consideration. Much more readable, and the errors are comprehensible.

[A] What is SFINAE, and what replaced it?

"Substitution Failure Is Not An Error" — a template that fails to substitute is silently removed from overload resolution rather than being a hard error. Used pre-C++20 to constrain templates via `enable_if`.

Concepts (C++20) do the same thing readably:

```
template <typename T, typename = std::enable_if_t<std::is_integral_v<T>>>   // old
void f(T x);

template <std::integral T>                                                  // new
void f(T x);
```

Know the difference and why concepts won: readability and error messages.

[A] How would you use a type trait as a correctness guard?

```
static_assert(std::is_trivially_copyable_v<Tick>);
```

From the assessment: the lock-free path copies a `Tick` bitwise — no constructor, no allocation, no destructor. If someone later adds a `std::string` symbol field, that silently breaks. The assert turns it into a compile error **at the line that caused it**, rather than a subtle runtime bug found at 3am.

That's the useful pattern: encode the assumption your algorithm depends on.

Section 7 — Performance and Systems

[I] How would you make this struct cache-friendly?

Order members largest-to-smallest to minimise padding. Keep hot fields together and move cold fields out. Consider splitting hot and cold into separate arrays. `alignas(64)` anything written by different threads.

And check the size — if it crosses a cache line boundary you pay two fetches for one object.

[I] AoS vs SoA?

Depends entirely on access pattern. Touch most fields of one object → array of structs. Touch one field across many objects → struct of arrays, because it's contiguous and vectorisable.

From the assessment, both directions appeared: the ranking engine went from `[stock][day]` to `[day][stock]` for a **3× win**, because the work was cross-sectional — 500 scattered 16-byte reads became two contiguous streams. But the position tracker deliberately used one struct per instrument, because quantity, average price and notional are always written together.

[A] Walk me through how you'd profile a slow C++ program.

`perf stat` first — three numbers tell you the shape immediately: **IPC** (low means stalling), **LLC-load-misses** (memory bound), **branch-misses** (mispredict bound). Then `perf record` / `report` to attribute by function. `perf c2c` for false sharing on threaded code. `-fopt-info-vec-missed` to check a loop actually vectorised rather than assuming.

Report **percentiles, not means** — a mean hides exactly the tail that matters.

And one habit: check that component timings add up to the end-to-end number. When they don't, the benchmark is lying. That mismatch is how I found a memory-layout bug in my own code during the assessment — the pieces summed to 4 µs but end-to-end was 18 µs.

[A] What benchmarking mistakes have you made?

Three, all of which made the code look *better* than it was:

- **Constant input** — the engine looked 4× faster because the sort's input arrived already sorted.
- **Hot repeated data** — re-sorting one array in a loop looked 5× faster, because the branch predictor memorised the comparison pattern. Cycling 256 distinct datasets revealed the truth.
- **A harness bug** that constructed a new object per iteration, inflating the baseline 3×.

Also: take the **minimum** of repeated runs, not the mean — it's least contaminated by scheduling noise. And use a `volatile` sink or the result gets optimised away entirely.

[A] When would you not parallelise something?

When it isn't the bottleneck — which is more often than people expect.

From the assessment: I was asked to parallelise a 500-element loop. It was **306 ns of an 18,000 ns day — 1.7% of the work.** Amdahl caps the achievable speedup at 1.017×, while creating and joining threads costs 14–25 µs. Measured, it came out **46–82× slower**.

The real win was questioning the requirement: the spec asked for top gainers and losers, not a total order, so `partial_sort` gave **2.4×** with no threads at all.

The deeper risk is that naive parallelism is an *invisible* failure — the code looks more sophisticated, the review passes, and the 90% that actually costs money never gets examined.

Section 8 — Floating Point

[B] Why is `0.1 + 0.2 != 0.3`?

Neither 0.1 nor 0.2 is exactly representable in binary — they're repeating fractions. The accumulated rounding lands at `0.30000000000000004`.

[I] How should you compare two floats?

Depends on intent. Values produced by *arithmetic*: compare with a relative tolerance. Values that must be **bit-identical** — a copy, a serialisation round-trip, a reimplementation you're validating — compare exactly, or compare `.hex()`.

From the assessment: the parity tests compare hex representations deliberately, because "close enough" is exactly the failure mode they exist to catch.

[I] What do you know about NaN?

Produced by `0.0/0.0`, `inf - inf`, `sqrt(-1)`. **Every ordered comparison involving NaN is false**, including `NaN == NaN`. That's why `x != x` is the standard NaN test.

Two practical consequences:

```
if (!(price > 0.0))    // catches zero, negatives, AND NaN
if (price <= 0.0)      // MISSES NaN, because that's also false
```

Equivalent for normal numbers, opposite for NaN. Choosing deliberately is the point.

And NaN in a sort comparator is UB, because it breaks strict weak ordering.

[A] What is FMA contraction and why might it matter?

A fused multiply-add computes `a*b + c` with a **single rounding** instead of two. It's faster and *more* accurate — which is precisely why it breaks bit-for-bit reproduction of a two-step computation.

Measured on the assessment, 3,000 fills against a Python reference:

Build	Fills differing
`-O2`	0
`-O2 -march=native`	**1,781**
`-Ofast -march=native`	1,659

The subtlety: GCC's default is already `-ffp-contract=fast`. Plain `-O2` passes only because baseline x86-64 has **no FMA instruction** — the compiler is allowed to fuse but has nothing to fuse with. Add `-march=native` and it appears. And FMA is **baseline on ARMv8**, so the same code would contract by default on Graviton or Apple Silicon.

So parity survived on two coincidences neither of which anyone chose. Setting `-ffp-contract=off` explicitly makes it depend on a stated decision.

[A] When would you use fixed point instead of floating point?

When values are exact multiples of a known unit — prices quoted in ticks, currency in paise.

Integers are exact under addition and subtraction, so no drift accumulates however long the session runs. No NaN, no infinity, no denormals. And faster.

Cost: you manage the scale yourself, so a units error becomes a silent 10,000× rather than a visible one. Division still rounds, so you decide where the remainder goes.

Measured: 200,000 accumulating float additions of 1/3 drifted from `0.3333333333333333` to `0.33333333333395226`. Fixed point: exact.

Section 9 — Short Coding Problems

These come up as whiteboard or screen-share exercises. Talk through the tradeoff, not just the code.

Reverse a linked list in place.

```
Node* reverse(Node* head) {
    Node* prev = nullptr;
    while (head) {
        Node* next = head->next;
        head->next = prev;
        prev = head;
        head = next;
    }
    return prev;
}
```

Implement a fixed-size ring buffer.

```
class Ring {
public:
    explicit Ring(size_t cap) : buf_(cap), head_(0), count_(0) {}

    void push(int v) {
        buf_[head_] = v;
        head_ = (head_ + 1) % buf_.size();
        if (count_ < buf_.size()) ++count_;
    }
    // i = 0 is the oldest
    int get(size_t i) const {
        size_t n = buf_.size();
        return buf_[(head_ + n - count_ + i) % n];   // + n stops unsigned underflow
    }
private:
    std::vector<int> buf_;
    size_t head_, count_;
};
```

Say the `+ n` out loud — `head_ - count_` can go "negative" on an unsigned type and wrap to a huge number, producing a valid-looking but wrong index. Silent corruption, not a crash.

Find the two numbers in an array that sum to a target.

Hash set, O(n): for each element check whether `target - x` has been seen.

The follow-up you should volunteer: if the array is sorted, two pointers from both ends is O(n) with no allocation and far better cache behaviour. For small n, that beats the hash set outright.

Implement a thread-safe counter — then justify it.

```
std::atomic<long> counter{0};
counter.fetch_add(1, std::memory_order_relaxed);
```

Relaxed is correct here because we only need atomicity — we don't care about ordering relative to other memory. Don't reach for a mutex, and don't default to `seq_cst`.

Implement `unique_ptr`.

```cpp
template <typename T>
class UniquePtr {
public:
    explicit UniquePtr(T* p = nullptr) : ptr_(p) {}
    ~UniquePtr() { delete ptr_; }

    UniquePtr(const UniquePtr&) = delete;                // no copying
    UniquePtr& operator=(const UniquePtr&) = delete;

    UniquePtr(UniquePtr&& o) noexcept : ptr_(o.ptr_) { o.ptr_ = nullptr; }
    UniquePtr& operator=(UniquePtr&& o) noexcept {
        if (this != &o) { delete ptr_; ptr_ = o.ptr_; o.ptr_ = nullptr; }
        return *this;
    }

    T& operator*()  const { return *ptr_; }
    T* operator->() const { return ptr_; }
    T* get()        const { return ptr_; }
    T* release()          { T* p = ptr_; ptr_ = nullptr; return p; }
private:
    T* ptr_;
};
```

Points to mention: copy deleted, moves `noexcept` (so `vector` will move rather than copy on reallocation), self-assignment check, and that the real one also carries a deleter and a separate array specialisation.

Section 10 — Questions to Ask Them

Having good questions matters more than people think — it's the clearest signal of genuine interest.

- What does the tick-to-trade path look like, and where does the time actually go?
- How do you test strategy changes before they go live?
- What does the build and deploy cycle look like during market hours?
- How much of the stack is C++ versus Python, and where's the boundary?
- What's the hardest bug the team has hit in the last year?
- How is work split between research and production engineering?
- What would you want me to have done well in the first three months?

The last one is the most useful — it tells you what they actually measure, and it's hard to answer vaguely.